

## **PROJECT REPORT**

# **ScaleOn Study Portal**

***Full-Stack Internship Management and Learning Portal***

***Prepared from the public project repository***

***[github.com/amangovindrao/ScaleOn\\_Study\\_Portal](https://github.com/amangovindrao/ScaleOn_Study_Portal)***

***Repository reviewed: September 2026***

# Abstract

*ScaleOn Study Portal is a full-stack web application designed to manage internship programs and provide a structured learning environment for interns. The system combines administrative management, role-based access control, learning modules, assignments, progress tracking, sessions, support tickets, and audit logging in a single platform. The repository documents a Next.js 16 and React 19 frontend, a Node.js 22 / Express.js backend written in TypeScript, and a PostgreSQL database accessed through Prisma ORM. Authentication uses JWT access and refresh tokens with HTTP-only cookies, while the architecture includes rate limiting, CSRF protection, security headers, input validation, and audit trails. The database documentation describes 29 tables covering identity, roles, internship enrollment, learning, assignments, activity, sessions, notifications, and support. The project is structured as separate frontend and backend applications with detailed architecture, API, database, and setup documentation.*

## 1. Introduction

*Internship programs commonly require coordination of users, batches, roles, learning content, assignments, progress, attendance, communication, and administrative controls. Managing these activities through separate tools can make tracking difficult and can reduce visibility for both administrators and interns.*

*ScaleOn Study Portal addresses this requirement by bringing internship lifecycle management and learning activities into one web platform. The project description identifies the core purpose as managing intern accounts, batches, roles, and learning content while allowing interns to track progress, complete modules, submit assignments, and earn XP. The repository also states that there is no public registration; intern accounts are created by administrators.*

## 2. Problem Statement

*A scalable internship program needs a centralized system that can securely manage participants, control access, organize learning activities, monitor progress, process assignments, and preserve an auditable history of important actions. The problem is therefore to design a maintainable full-stack application that separates administrative operations from intern-facing learning workflows while supporting secure authentication and database-driven permissions.*

## 3. Objectives

- *Provide centralized management of interns, batches, roles, and internship enrollment.*
- *Provide a dedicated learning environment with phases, modules, progress tracking, XP, and streaks.*
- *Support assignment creation, submission review, grading, status changes, and feedback.*
- *Implement secure authentication and session management using access and refresh tokens.*
- *Enforce database-driven role-based access control and granular permissions.*
- *Maintain login history, activity logs, and administrative audit logs.*

- *Provide support-ticket and live-session functionality for internship operations.*
- *Design the database and service architecture for growth from small cohorts to large intern populations.*

## 4. Scope of the Project

*The project scope can be grouped into the following major areas:*

| Area                         | Scope                                                                                             |
|------------------------------|---------------------------------------------------------------------------------------------------|
| Identity & Authentication    | Admin/intern login, password reset, JWT access tokens, rotating refresh tokens, session tracking. |
| User & Internship Management | Intern CRUD, account status, batch assignment, role transfer, activation/suspension, profiles.    |
| Learning Management          | Learning phases, modules, module progress, XP, levels, streaks and daily activity.                |
| Assignments                  | Assignment creation/editing/deletion, submissions, review, scores, feedback and status.           |
| Administration               | Roles, permissions, batches, sessions and administrative controls.                                |
| Communication & Support      | Notifications, live sessions, attendance and support tickets.                                     |
| Audit & Security             | Login history, activity logs, audit logs, rate limiting, CSRF and security headers.               |

## 5. Technology Stack

| Component      | Technology                                                       |
|----------------|------------------------------------------------------------------|
| Frontend       | Next.js 16, React 19, TypeScript, Tailwind CSS 4                 |
| Backend        | Node.js 22, Express.js, TypeScript                               |
| Database       | PostgreSQL                                                       |
| ORM            | Prisma                                                           |
| Authentication | JWT access + refresh tokens, HTTP-only cookies                   |
| Security       | bcrypt, Helmet, HPP, rate limiting, brute-force protection, CSRF |
| UI & Animation | Framer Motion, Lucide React                                      |
| Validation     | Zod                                                              |
| Architecture   | Separated frontend/backend with REST-style APIs                  |

## 6. System Architecture

*The architecture is organized into a frontend application, a backend API server, and a PostgreSQL database. The frontend uses Next.js App Router and reusable React components. The backend uses Express.js with TypeScript and Prisma as the data-access layer. Protected backend routes apply authentication and permission middleware before business operations are executed.*

*High-level flow:*

- *User → Next.js frontend*
- *Frontend → REST API / authentication layer*
- *Express middleware → authentication, authorization, validation and security checks*
- *Application services/modules → business logic*
- *Prisma ORM → PostgreSQL database*
- *Database → logs, learning data, assignments, sessions and user/internship records*

*The repository's architecture document also identifies planned infrastructure areas such as file storage, email, real-time notifications, caching, search, deployment, and observability as future or pending decisions.*

## **7. Functional Modules**

### **Authentication Module**

*Handles login, logout, token refresh, password reset and account/session controls.*

### **Intern Management**

*Allows administrators to create and manage intern accounts, statuses, batches, roles and transfers.*

### **Role & Permission Management**

*Uses database-driven roles and granular permission groups instead of hardcoded access rules.*

### **Learning Module**

*Organizes learning into phases and modules and records completion, score, time, XP and streak information.*

### **Assignment Module**

*Supports assignment lifecycle management and submission review with scores, feedback and status decisions.*

### **Session Management**

*Tracks active/past sessions and allows administrators to terminate sessions.*

### **Live Sessions**

*Stores scheduled/live/completed sessions and intern attendance.*

### **Support**

*Provides support tickets and ticket messages for intern requests.*

### **Notifications**

*Provides in-app notification records for system, account, credential, announcement, reminder and certificate events.*

**Audit & Activity**

*Records login attempts, user activity and administrative mutations for traceability.*

**8. Authentication and Security**

*Security is a major part of the documented architecture. Authentication uses a short-lived JWT access token and a rotating refresh-token mechanism. Tokens are delivered through HTTP-only cookies, and refresh tokens are stored as hashes rather than raw token values.*

- *JWT access tokens are documented with a 15-minute lifetime.*
- *Refresh tokens are documented with a 7-day lifetime, extended to 30 days with remember-me.*
- *Passwords are hashed using bcrypt.*
- *Login rate limiting is documented as five attempts per 15 minutes.*
- *Brute-force protection can lock an account after five failed attempts.*
- *CSRF protection is included.*
- *Helmet provides security-related HTTP headers.*
- *HPP is used to reduce HTTP parameter pollution risks.*
- *Zod validates request bodies.*
- *Prisma parameterized queries are used to reduce SQL injection risk.*
- *LoginHistory, ActivityLog and AuditLog provide operational traceability.*

**9. Role-Based Access Control**

*The system uses database-driven RBAC. Roles have privilege levels and are associated with granular permissions. The documented roles are Super Admin, Admin, Mentor and Intern. The architecture allows permissions to be grouped for intern management, administration, roles, batches, content, certificates, sessions and authentication.*

| Role        | Level     | Documented responsibility                                          |
|-------------|-----------|--------------------------------------------------------------------|
| Super Admin | Level 100 | Full platform control; system role cannot be deleted.              |
| Admin       | Level 50  | Manage interns, content, batches and roles.                        |
| Mentor      | Level 25  | Planned/future role for guiding interns and reviewing progress.    |
| Intern      | Level 10  | Learning access, progress tracking and certificate-related access. |

**10. Database Design**

*The database is PostgreSQL with Prisma ORM. The repository documentation lists 29 tables. The design uses UUID primary keys, indexes on foreign keys/status/frequently queried columns, soft deletion for user accounts, and database-driven permissions.*

| <b>Group</b>                       | <b>Representative tables</b>                                                          |
|------------------------------------|---------------------------------------------------------------------------------------|
| <b>Identity</b>                    | <b>UserAccount, Admin, Intern, InternProfile</b>                                      |
| <b>Authorization</b>               | <b>Role, Permission, RolePermission</b>                                               |
| <b>Internship</b>                  | <b>InternshipRole, Batch, InternshipEnrollment</b>                                    |
| <b>Security &amp; Audit</b>        | <b>Session, RefreshToken, PasswordResetToken, LoginHistory, ActivityLog, AuditLog</b> |
| <b>Learning</b>                    | <b>LearningPhase, LearningModule, ModuleProgress, InternStreak, DailyActivity</b>     |
| <b>Assignments &amp; Sessions</b>  | <b>Assignment, AssignmentSubmission, LiveSession, LiveSessionAttendee</b>             |
| <b>Support &amp; Notifications</b> | <b>SupportTicket, TicketMessage, Notification, Setting</b>                            |

# 11. API Design

*The documented API follows REST-style resource naming with GET, POST, PUT, PATCH and DELETE methods. Protected endpoints use authentication middleware and permission checks. JSON is used for request/response bodies, with documented status codes including 200, 201, 400, 401, 403, 404 and 500.*

*Major documented API modules:*

- */auth*
- */users*
- */admins*
- */interns*
- */roles*
- */batches*
- */internship-roles*
- */sessions*
- */profile*

# 12. Frontend Structure

*The frontend uses Next.js App Router. Authentication pages are separated from admin and intern areas. The repository structure includes reusable UI components and frontend utility libraries.*

- *app/(auth)– login, forgot-password and reset-password flows*
- *app/admin– dashboard and administrative modules*
- *app/intern– intern dashboard, learning, leaderboard, assignments and support*
- *app/onboarding– first-login onboarding wizard*
- *components– shared UI components such as navigation, cards, modals, badges and inputs*
- *lib– API client, authentication context, hooks and utility functions*

## 13. Assignment Management Module

*The repository includes an admin assignment management module. It provides an overview of assignments, search and filtering, create/edit/delete workflows, and submission review.*

*Administrators can inspect submissions, assign scores within the maximum score, write feedback, and change review status.*

- *Metrics for total tasks and submissions, pending reviews and maximum-score information.*
- *Search by assignment title or description.*
- *Filtering by learning module and due-date status.*
- *Assignment creation and editing through modal forms.*
- *Submission review with score, feedback and status.*
- *Dedicated components for tables, forms, submissions, review dialogs, filters, search and empty/loading states.*

## 14. Installation and Execution

*The repository's quick-start documentation separates dependency installation and runtime execution for the backend and frontend. A backend environment file is required for the database URL and security secrets.*

1. *Clone the repository.*
2. *Install backend dependencies with `npm install` inside `/backend`.*
3. *Install frontend dependencies with `npm install` inside `/frontend`.*
4. *Create backend `.env` from the supplied environment template and configure required secrets.*
5. *Generate the Prisma client.*
6. *Run the backend development server.*
7. *Run the frontend development server.*
8. *Open the admin or intern login route in a browser.*

## 15. Strengths of the Current Design

- *Clear separation between frontend, backend, and database responsibilities.*
- *Strong emphasis on authentication, authorization and auditability.*
- *Database-driven RBAC allows permissions to evolve without rewriting authorization logic.*
- *The database model covers both internship administration and learning operations.*
- *The architecture is modular, making it easier to add features independently.*
- *The repository contains dedicated architecture, database, API, setup and changelog documentation.*

- *Use of TypeScript across frontend and backend improves type consistency.*
- *PostgreSQL and Prisma provide a structured relational data layer suitable for transactional workloads.*

## **16. Limitations / Areas Marked for Future Work**

*The following items are explicitly identified in the repository documentation as pending, future, or to-be-implemented rather than being treated as completed functionality:*

- *Production deployment strategy is documented but marked as future/TBD.*
- *Observability such as health checks, metrics, structured logging and APM is marked for future implementation.*
- *File storage for profile photos, resumes and certificates is still a future architecture decision.*
- *Email infrastructure is listed as a future choice among providers.*
- *Real-time notification infrastructure is listed as a future decision.*
- *Redis caching/rate-limit/session support is listed as a future option.*
- *Advanced search using Algolia or Elasticsearch is listed as a future option.*
- *Mentor-specific functionality and some content/certificate permission areas are documented as future-oriented.*

## **17. Testing and Quality Considerations**

*This report is based on repository documentation and source-tree information rather than an independent execution or penetration test. Therefore, the security and functional features described above should be understood as documented project capabilities/design elements, not as independently verified production results.*

*Recommended validation activities before production deployment:*

- *Unit-test authentication, authorization and service-layer business rules.*
- *Integration-test critical API endpoints against a test PostgreSQL database.*
- *Test RBAC combinations and privilege escalation scenarios.*
- *Test refresh-token rotation, logout and session termination.*
- *Run dependency and static-security scans.*
- *Test assignment grading boundaries and invalid inputs.*
- *Load-test intern search, dashboards and learning-progress endpoints.*
- *Verify audit-log completeness for critical administrative mutations.*

## **18. Future Enhancements**

- *Add production observability with health checks, metrics, structured logs and APM.*
- *Integrate scalable file storage for resumes, certificates and profile media.*

- *Add real-time notifications and event-driven updates.*
- *Introduce Redis where caching or distributed rate limiting becomes necessary.*
- *Add advanced search and filtering for large intern populations.*
- *Expand mentor workflows for guidance, reviews and progress monitoring.*
- *Introduce richer analytics dashboards for administrators and internship performance.*
- *Add automated deployment and CI/CD pipelines with security checks.*

## 19. Conclusion

*ScaleOn Study Portal is a structured full-stack platform intended to centralize internship administration and learning. Its architecture combines a modern Next.js frontend, a TypeScript/Express backend, PostgreSQL with Prisma, and a security-oriented authentication and authorization model. The project goes beyond a simple study portal by including intern lifecycle management, database-driven RBAC, learning progress, assignments, sessions, support, notifications and auditability. The repository also documents a clear modular structure and several planned infrastructure improvements. Overall, the project provides a substantial foundation for an internship management and learning platform, while the documented future work indicates the areas that would need further implementation and operational validation for a production-scale deployment.*

## 20. References

1. *ScaleOn Study Portal GitHub repository:*  
[https://github.com/amangovindrao/ScaleOn\\_Study\\_Portal](https://github.com/amangovindrao/ScaleOn_Study_Portal)
2. *Repository README.md – project overview, features, technology stack and setup.*
3. *Repository ARCHITECTURE.md – system architecture, authentication, RBAC, security and deployment strategy.*
4. *Repository DATABASE.md – PostgreSQL/Prisma design and 29-table database documentation.*
5. *Repository API\_DOCUMENTATION.md – REST API modules and endpoint documentation.*

**End of Report**